

factor_product(A, B) and factor_marginalize(factor, var)

`factor_product()` combines two factors over the union of their variables. Assignments in the joint scope are projected back to the scopes of the two input factors using the provided assignment/index conversion utilities, after which the corresponding entries are multiplied.

`factor_marginalize()` removes selected variables by summing all assignments that map to the same assignment of the remaining variables.

$$\phi(X_R) = \sum_{X_M} \phi(X_R, X_M)$$

factor_evidence(factor, evidence)

conditions a factor on the observed variables. Assignments inconsistent with the evidence are removed, and the observed variables are removed from the returned factor scope. If all variables of a factor are observed, an empty factor is returned.

Junction-tree construction

`_get_jt_clique_and_edges(nodes, edges)`: first represents the MRF as an undirected graph and triangulates it to obtain a chordal graph. Maximal cliques are then extracted using `nx.find_cliques()`. A clique graph is formed where overlapping cliques are connected, with separator cardinality used as the edge weight:

$$S_{ij} = C_i \cap C_j, \quad w_{ij} = |S_{ij}|$$

The junction tree is obtained by taking the maximum spanning tree. Each junction-tree edge is stored in both directions since messages are passed in both directions later. This should follow the junction-tree construction described in Lecture 5.

`_get_clique_factors(jt_cliques, factors)`: assigns each original factor to a clique containing its entire variable scope. Factors assigned to the same clique are multiplied using `factor_product()`. Each factor is assigned only once to avoid double counting.

Evidence update

`_update_mrf_w_evidence(all_nodes, evidence, edges, factors)`: removes observed nodes from the query set and removes edges incident to them. Every factor is conditioned using `factor_evidence()`, and factors whose complete scope has been observed are discarded. Thus the junction tree is constructed only over the remaining query variables.

`_get_clique_potentials(jt_cliques, jt_edges, jt_clique_factors)`: performs a sum-product message passing over the junction tree. A message from clique C_i to C_j combines the local clique factor with all incoming messages except from C_j , then marginalizes variables outside the separator:

$$m_{i \rightarrow j} = \sum_{C_i \setminus S_{ij}} \phi_i \prod_{k \in N(i) \setminus j} m_{k \rightarrow i}$$

Messages are computed recursively and cached so that each directed message is only evaluated once. The final clique potential is obtained by multiplying the local factor with all incoming messages.

_get_node_marginal_probabilities(query_nodes, cliques, clique_potentials): uses the hint that cliques may have different sizes. I select the smallest clique containing each query node, which reduces the number of variables that need to be marginalized out. The remaining unary factor is then normalized:

$$p(X_i) = \frac{\tilde{p}(X_i)}{\sum_{x_i} \tilde{p}(x_i)}$$